

Roteiro de Apresentação — Desafio Organização Financeira

Tempo sugerido total: 8-10 minutos. Ajuste conforme o tempo que vocês tiverem.

1. Abertura — o problema (1 min)

O que falar:

"Nosso desafio era criar uma solução capaz de registrar, organizar, analisar e consultar dados de gastos pessoais de forma eficiente ao longo do tempo. Decidimos aplicar isso num caso real: uma família, onde os pais registram os gastos e podem analisar se sobra dinheiro no orçamento."

Por que começar assim: contextualiza o problema antes de entrar em código — mostra que vocês entenderam o desafio antes de sair implementando.

2. Processo de raciocínio — guiding questions (1-2 min)

O que falar:

"Antes de programar, definimos perguntas norteadoras (guiding questions) pra guiar as decisões do projeto: o que conta como gasto, quem vai consultar os dados, com que frequência seriam registrados, e por que usar banco de dados em vez de planilha comum."

Dica: se tiver o board do Miro, mostre a tela rapidamente aqui. Isso demonstra processo, não só resultado.

3. Por que banco de dados relacional (1 min)

O que falar:

"Escolhemos um banco relacional em vez de uma planilha porque ele evita repetição de dados e garante consistência. Por exemplo: em vez de escrever o nome da pessoa em toda linha de gasto, cadastramos a pessoa uma única vez, e cada gasto só referencia esse cadastro. Isso evita erros como 'André', 'andre' e 'André ' sendo tratados como pessoas diferentes."

Ferramenta usada: PostgreSQL, definido como requisito do desafio.

4. Modelo relacional — como as tabelas se conectam (2 min)

O que mostrar: o diagrama das duas tabelas (pode desenhar rápido no quadro ou mostrar o

Excalidraw/ERD).

O que falar:

"Criamos duas tabelas: `usuarios`, que cadastra cada pessoa da família, e `gastos`, que registra cada gasto individual. A conexão entre elas acontece através de uma **chave primária** e uma **chave estrangeira**."

Explicando o código na tela

sql

```
CREATE TABLE usuarios(  
  id SERIAL PRIMARY KEY,  
  nome VARCHAR(50) NOT NULL,  
  email VARCHAR(100) UNIQUE NOT NULL,  
  data_criacao TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

"Aqui, `id SERIAL PRIMARY KEY` cria um identificador único e automático pra cada pessoa — como um RG. `UNIQUE` no e-mail impede cadastro duplicado. E `DEFAULT CURRENT_TIMESTAMP` preenche a data de cadastro sozinho, sem precisarmos informar."

sql

```
CREATE TABLE gastos(  
  id SERIAL PRIMARY KEY,  
  usuario_id INTEGER NOT NULL REFERENCES usuarios(id),  
  valor NUMERIC(10,2) NOT NULL,  
  descricao TEXT NOT NULL,  
  data DATE NOT NULL  
);
```

"A parte mais importante é essa linha: `usuario_id INTEGER NOT NULL REFERENCES usuarios(id)`. Isso é a **chave estrangeira** — ela diz que todo gasto precisa estar vinculado a uma pessoa que já existe na tabela `usuarios`. Se tentarmos inserir um gasto com um `usuario_id` que não existe, o próprio banco recusa automaticamente, com um erro de violação de chave estrangeira. Isso garante que nunca vamos ter um gasto 'órfão', sem dono."

Se quiser demonstrar ao vivo (opcional, impressiona bastante):

sql

```
-- Isso vai dar erro de propósito, pra mostrar a proteção funcionando  
INSERT INTO gastos (usuario_id, valor, descricao, data)  
VALUES (999, 50.00, 'Teste', '2026-07-01');
```

5. Como os dados são registrados (1 min)

O que falar:

"Pra registrar um gasto, informamos o número de identificação da pessoa (não o nome escrito por extenso), o valor, a descrição e a data. Isso evita inconsistência de digitação."

sql

```
INSERT INTO usuarios (nome, email) VALUES
  ('Andre', 'pazinihoch@gmail.com'),
  ('Caua', 'caua@hotmail.com');

INSERT INTO gastos (usuario_id, valor, descricao, data) VALUES
  (1, 100.00, 'Gasolina', '2026-07-02'),
  (2, 52.60, 'Mercado', '2026-07-05');
```

6. Conceito técnico principal: o JOIN (2 min)

Esse é o momento de destacar o conceito mais importante do trabalho.

O que falar:

"Quando queremos consultar os dados, o número da pessoa sozinho não é legível — ninguém quer ver '1 gastou R\$ 100'. Pra trazer o nome de volta, usamos o comando `JOIN`, que junta as duas tabelas na hora da consulta."

sql

```
SELECT usuarios.nome, gastos.valor, gastos.descricao, gastos.data
FROM gastos
JOIN usuarios ON gastos.usuario_id = usuarios.id
WHERE usuarios.nome = 'Andre';
```

"O `JOIN` funciona assim: para cada gasto, ele olha o número (`usuario_id`), procura na tabela `usuarios` quem tem esse mesmo número como `id`, e junta as informações das duas tabelas numa única linha de resultado. É importante deixar claro que o `JOIN` **não altera nada no banco** — ele só existe no momento da consulta, sempre que precisamos visualizar dados combinados."

Frase de efeito pra fixar o conceito:

"Resumindo: guardamos os dados de forma separada e organizada, mas conseguimos visualizá-los juntos sempre que precisamos, sem nunca duplicar informação."

7. Análises feitas — os insights (2 min)

O que falar:

"A partir dessa estrutura, conseguimos responder perguntas reais sobre os gastos da família."

Exemplo 1 — Total por pessoa

sql

```
SELECT usuarios.nome, SUM(gastos.valor) AS total_gasto
FROM gastos
JOIN usuarios ON gastos.usuario_id = usuarios.id
GROUP BY usuarios.nome
ORDER BY total_gasto DESC;
```

"Aqui usamos `SUM` pra somar os valores, e `GROUP BY` pra separar essa soma por pessoa. Descobrimos que [FALAR O RESULTADO REAL DE VOCÊS AQUI]."

Exemplo 2 — Mês que mais gastou

sql

```
SELECT TO_CHAR(gastos.data, 'YYYY-MM') AS mes, SUM(gastos.valor) AS total_gasto
FROM gastos
GROUP BY mes
ORDER BY total_gasto DESC;
```

"Aqui identificamos qual mês teve maior gasto total, o que ajuda a entender se há sazonalidade nos gastos da família."

8. Conceito técnico avançado: DISTINCT ON (1-2 min)

Esse é um ótimo momento pra "impressionar", mostrando que foram além do básico.

O que falar:

"Um desafio que enfrentamos foi: como descobrir o mês de maior gasto de **cada** pessoa, sem uma pessoa 'roubar' o lugar da outra no resultado? Usar `LIMIT` sozinho não resolvia, porque ele corta o resultado geral, e não por grupo. A solução foi o `DISTINCT ON`."

sql

```
SELECT DISTINCT ON (usuarios.nome)
    usuarios.nome,
    TO_CHAR(gastos.data, 'YYYY-MM') AS mes,
    SUM(gastos.valor) AS total_gasto
FROM gastos
JOIN usuarios ON gastos.usuario_id = usuarios.id
GROUP BY usuarios.nome, TO_CHAR(gastos.data, 'YYYY-MM')
ORDER BY usuarios.nome, total_gasto DESC;
```

"O `DISTINCT ON (usuarios.nome)` garante uma linha por pessoa — especificamente, a **primeira** linha depois da ordenação. Como ordenamos por `total_gasto DESC` dentro de cada nome, essa primeira linha acaba sendo automaticamente o mês de maior gasto daquela pessoa. Isso é um recurso mais avançado, específico do PostgreSQL, que aprendemos justamente pra resolver esse problema real que apareceu durante o desenvolvimento."

9. Fechamento — conclusão e próximos passos (1 min)

O que falar:

"Com essa estrutura, a família consegue visualizar padrões de gasto, comparar meses, e entender melhor o orçamento — que era exatamente o objetivo do desafio: registrar, organizar, analisar e consultar dados de forma eficiente ao longo do tempo. Como próximo passo, poderíamos expandir o modelo incluindo categorias de gasto ou até uma tabela de renda, pra calcular automaticamente quanto sobra no orçamento."

Checklist rápido pra revisar antes da apresentação

- ☐ Testar se o banco/pgAdmin abre sem erro antes da apresentação
- ☐ Ter os dados de exemplo já inseridos (não inserir ao vivo, perde tempo)
- ☐ Rodar as queries antes, uma vez, pra garantir que funcionam sem erro
- ☐ Anotar os números reais dos resultados (total por pessoa, mês que mais gastou) pra falar de cabeça
- ☐ Decidir quem fala qual parte do roteiro, se forem apresentar em dupla/grupo
- ☐ Ter o modelo relacional (diagrama) pronto pra mostrar visualmente

Perguntas que o professor pode fazer (se preparem para essas)

Pergunta possível	Resposta sugerida
Por que não usar planilha em vez de banco de dados?	Evita repetição/inconsistência de dados; permite consultas mais poderosas e rápidas

Pergunta possível	Resposta sugerida
O que acontece se tentarem inserir um gasto de alguém que não existe?	O banco recusa automaticamente, por causa da Foreign Key (podem demonstrar ao vivo)
Por que separaram em duas tabelas em vez de uma só?	Normalização — evita repetir nome/email em cada linha de gasto
O que é JOIN, de forma simples?	Uma forma de juntar informações de duas tabelas relacionadas na hora da consulta, sem duplicar dados armazenados